Environment Semantics

Concepts of Programming Languages

CAS CS 320

Outline

- Discuss the difference between **dynamic** and **lexical** scoping
- Introduce **closures** as a way of implementing lexical scoping in the environment model
- Discuss **recursion** and closures

Variable Scoping

Two Major Concerns

Two Major Concerns

1. Are variables *mutable*?

Two Major Concerns

1. Are variables *mutable*?
2. How are variables *scoped*? (Dynamically or lexically?)

Two Major Concerns

1. Are variables *mutable*?

2. How are variables *scoped*? (Dynamically or lexically?)

OCaml variables are immutable, binding defined, and lexically scoped

Mutability

```
let x = 0
let f () =
  let x = 1 in
  ()
print_int x
```

Immutable (OCaml)

```
x = 0
def f():
    global x
    x = 1
print(x)
```

Mutable (Python)

Mutability

```
let x = 0
let f () =
  let x = 1 in
  ()
print_int x
```

Immutable (OCaml)

```
x = 0
def f():
    global x
    x = 1
print(x)
```

Mutable (Python)

Def. *(informal)* A variable is **mutable** if we are allowed to change its value after it has been declared

Scope

Scope

Def. *(Informal)* The **scope** of a variable binding is *when* and *where* a variable can be accessed

Scope

Def. *(Informal)* The **scope** of a variable binding is *when* and *where* a variable can be accessed

There are two standard paradigms: **dynamic** scoping and **lexical** scoping (static scoping)

Scope

Def. *(Informal)* The **scope** of a variable binding is *when* and *where* a variable can be accessed

There are two standard paradigms: **dynamic** scoping and **lexical** scoping (static scoping)

Warning. Scope is one of the most overused terms in computer science...

Dynamic Scoping

```
f() { x=23; g; }
g() { y=$x; }
f
echo $y
```

Bash

Dynamic Scoping

```bash
f() { x=23; g; }
g() { y=$x; }
f
echo $y
```

Bash

Dynamic scoping refers to when bindings are determined at runtime based on *computational context*

Dynamic Scoping

```bash
f() { x=23; g; }
g() { y=$x; }
f
echo $y
```

Bash

Dynamic scoping refers to when bindings are determined at runtime based on *computational context*

This is a *temporal view*, i.e., what a computation done beforehand which affected the value of a variable

Lexical Scoping

```python
x = 0
def f():
    x = 1
    return x
assert(f() == 1)
assert(x == 0)
```

Python

```ocaml
let x = 0
let f () =
  let x = 1 in
  x
let _ = assert (f () = 1)
let _ = assert (x = 0)
```

OCaml

Lexical Scoping

```python
x = 0
def f():
    x = 1
    return x
assert(f() == 1)
assert(x == 0)
```

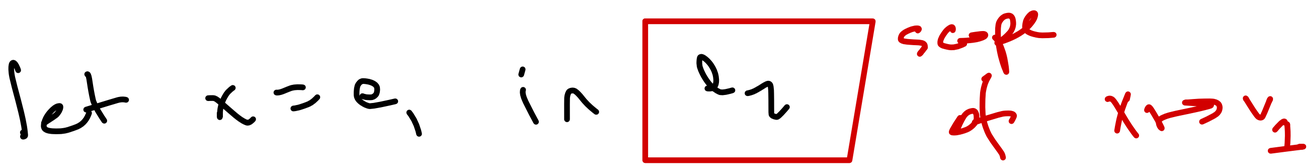
Python

```ocaml
let x = 0
let f () =
  let x = 1 in
  x
let _ = assert (f () = 1)
let _ = assert (x = 0)
```

OCaml

Lexical (static) scoping refers to the use of textual delimiters to define the scope of a binding

Lexical Scoping




```python
x = 0
def f():
    x = 1
    return x
assert(f() == 1)
assert(x == 0)
```

Python

```ocaml
let x = 0
let f () =
  let x = 1 in
  x
let _ = assert (f () = 1)
let _ = assert (x = 0)
```

OCaml

Lexical (static) scoping refers to the use of textual delimiters to define the scope of a binding

There are two common ways lexical scope is determined:

Lexical Scoping

```python
x = 0
def f():
    x = 1
    return x
assert(f() == 1)
assert(x == 0)
```

Python

```ocaml
let x = 0
let f () =
  let x = 1 in
  x
let _ = assert (f () = 1)
let _ = assert (x = 0)
```

OCaml

Lexical (static) scoping refers to the use of textual delimiters to define the scope of a binding

There are two common ways lexical scope is determined:

- The binding defines it's own scope (let-bindings)

Lexical Scoping

```python
x = 0
def f():
    x = 1
    return x
assert(f() == 1)
assert(x == 0)
```

Python

```ocaml
let x = 0
let f () =
  let x = 1 in
  x
let _ = assert (f () = 1)
let _ = assert (x = 0)
```

OCaml

Lexical (static) scoping refers to the use of textual delimiters to define the scope of a binding

There are two common ways lexical scope is determined:

- The binding defines it's own scope (let-bindings)
- A block defines the scope of a variable (python functions)

Tradeoffs

```
f() { x=23; g; }
g() { y=$x; }
f
echo $y
```

dynamic

vs.

```
let x = 0
let f () =
  let x = 1 in
  x
let _ = assert (f () = 1)
let _ = assert (x = 0)
```

lexical

Implementing dynamic scoping is *way* easier...

But **every modern PL** implements lexical scoping

The Environment Model

Motivation

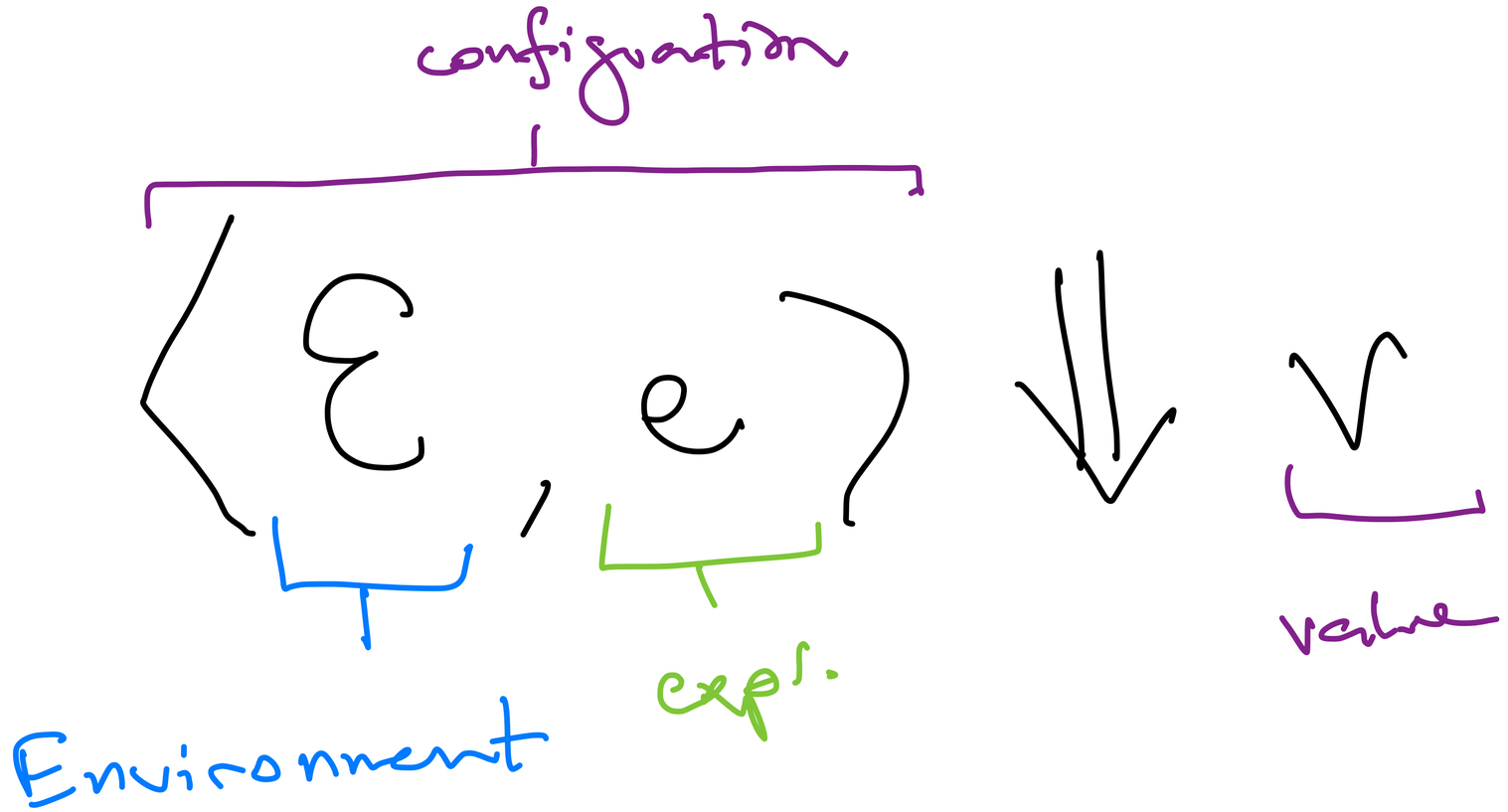
let x = v in (... x ... x ... x ... x ...)



(... v ... v ... v ... v ...)

Not very efficient

Environment Semantic Judgments



Environments

$\mathcal{E}[x \mapsto v](x) = v$

Mapping of var. to val.

$\{ x \mapsto 0, y \mapsto \text{true}, z \mapsto \lambda x.x \}$

var. binding

shadows existing binding

$\mathcal{E}$	environment
$\mathcal{E}[x \mapsto v]$	add $x \mapsto v$ in $\mathcal{E}$
$\mathcal{E}(x)$	value x is bound to in $\mathcal{E}$

Syntax (λ Calculus +)

$$
\begin{aligned}
e ::= \quad & \lambda x.e \mid e\,e \mid x \\
\mid \quad & \texttt{let } x = e \texttt{ in } e \quad \} \text{ let-expr.} \\
\mid \quad & \texttt{true} \mid \texttt{false} \quad \} \text{ Booleans}
\end{aligned}
$$

Environment Semantics (λ Calculus +)

$v ::= \lambda x.e \mid true \mid false$

$e ::= \lambda x.e \mid ee \mid x$
$\mid$ let $x = e$ in e } let-expr.
$\mid true \mid false$ } Booleans

$$\frac{}{\langle \mathcal{E}, \lambda x.e \rangle \Downarrow \lambda x.e}\ (fun)$$

$$\frac{}{\langle \mathcal{E}, true \rangle \Downarrow true}\ (true)$$

$$\frac{}{\langle \mathcal{E}, false \rangle \Downarrow false}\ (false)$$

$$\frac{}{\langle \mathcal{E}, x \rangle \Downarrow \mathcal{E}(x)}\ (var)$$

$$\frac{\langle \mathcal{E}, e_1 \rangle \Downarrow v_1 \qquad \langle \mathcal{E}[x \mapsto v_1], e_2 \rangle \Downarrow v_2}{\langle \mathcal{E}, \text{let } x = e_1 \text{ in } e_2 \rangle \Downarrow v_2}\ (let)$$

$$\frac{\langle \mathcal{E}, e_1 \rangle \Downarrow \lambda x.e \qquad \langle \mathcal{E}, e_2 \rangle \Downarrow v_2 \qquad \langle \mathcal{E}[x \mapsto v_2], e \rangle \Downarrow v}{\langle \mathcal{E}, e_1\, e_2 \rangle \Downarrow v}\ (app)$$

Back to Scoping

```
let x = true in
let f = λy.x in
let x = false in
f x
```

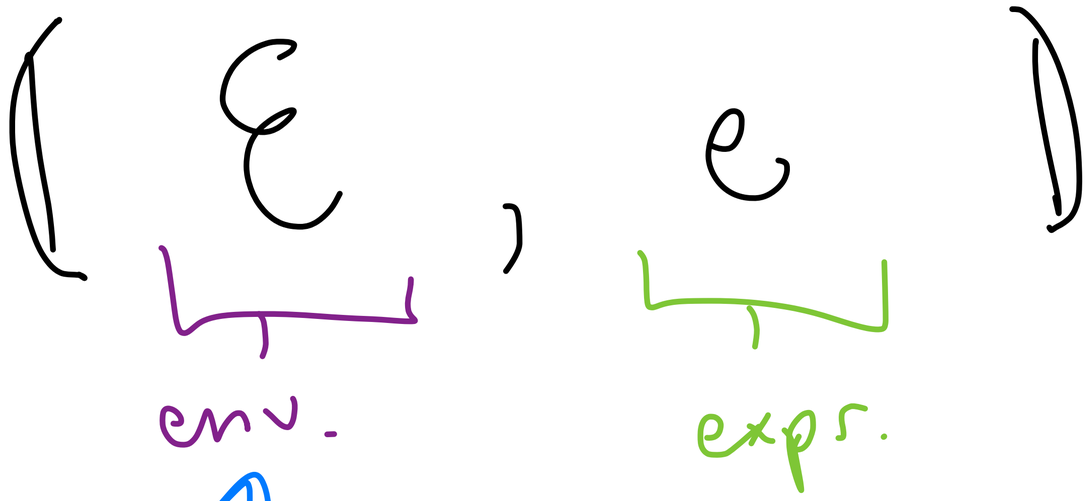
dynamically scoped

$$\frac{\dfrac{}{\langle \mathcal{E}', f \rangle \Downarrow \lambda y.x}\,\text{var} \qquad \dfrac{}{\langle \mathcal{E}', x \rangle \Downarrow \text{false}}\,\text{var} \qquad \dfrac{}{\langle \mathcal{E}'[x \mapsto \text{false}], x \rangle \Downarrow \text{false}}\,\text{var}}{\mathcal{E}'\{x \mapsto \text{false}, f \mapsto \lambda y.x\}}$$

$$\frac{\langle \mathcal{E}, \text{false} \rangle \Downarrow \text{false} \qquad \langle \mathcal{E}[x \mapsto \text{false}], f\ x \rangle \Downarrow \text{false}}{\langle \mathcal{E}\{x \mapsto \text{true}, f \mapsto \lambda y.x\}, \text{let } x = \text{false in } f\ x \rangle \Downarrow \text{false}}$$

Closures

Closures



"snapshot" of env. at a given time

functions can "remember" the env. when they were defined.

Values (λ Calculus +)

$$v ::= true \mid false \mid (\!| \mathcal{E}, \lambda x.e |\!)$$

not an expression

Semantics Again (λ Calculus +)

$v ::= true \mid false \mid (\mathcal{E}, \lambda x.e)$

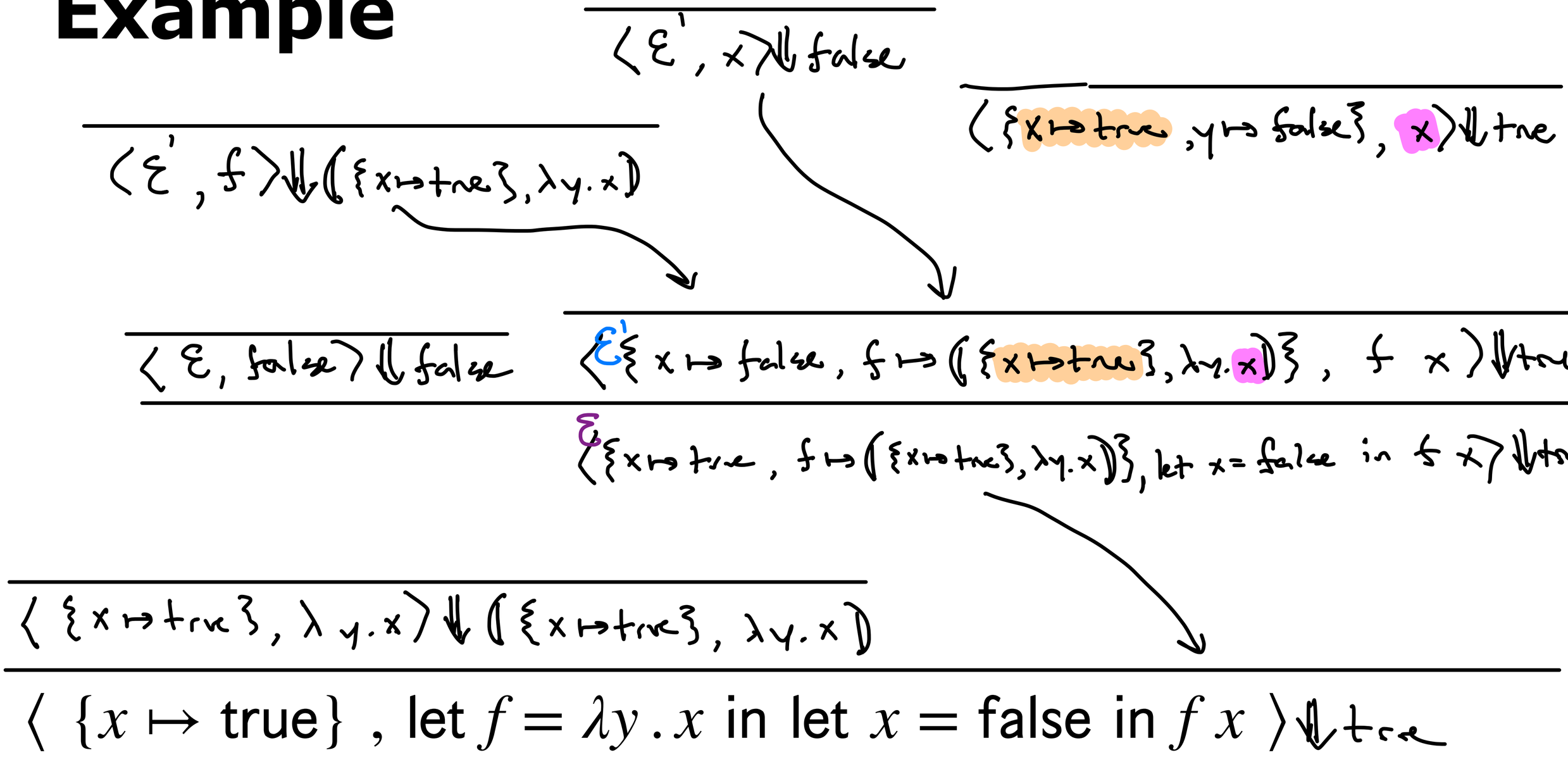
not an expression

$$\frac{}{\langle \mathcal{E}, \lambda x.e \rangle \Downarrow (\mathcal{E}, \lambda x.e)} \text{(fun)}$$

$$\frac{\langle \mathcal{E}, e_1 \rangle \Downarrow (\mathcal{E}', \lambda x.e) \quad \langle \mathcal{E}, e_2 \rangle \Downarrow v_2 \quad \langle \mathcal{E}'[x \mapsto v_2], e \rangle \Downarrow v}{\langle \mathcal{E}, e_1 e_2 \rangle \Downarrow v}$$

(everything else is the same)

Example


⟨ε', x⟩ ⇓ false
⟨ε', f⟩ ⇓ (⟦{x ↦ true}, λy.x⟧)
⟨{x ↦ true, y ↦ false}, x⟩ ⇓ true
⟨ε, false⟩ ⇓ false
ε'{x ↦ false, f ↦ (⟦{x ↦ true}, λy.x⟧)}, f x⟩ ⇓ true
ε{x ↦ true, f ↦ (⟦{x ↦ true}, λy.x⟧)}, let x = false in f x⟩ ⇓ true
⟨{x ↦ true}, λy.x⟩ ⇓ (⟦{x ↦ true}, λy.x⟧)
⟨ {x ↦ true} , let f = λy . x in let x = false in f x ⟩ ⇓ true

Recursion

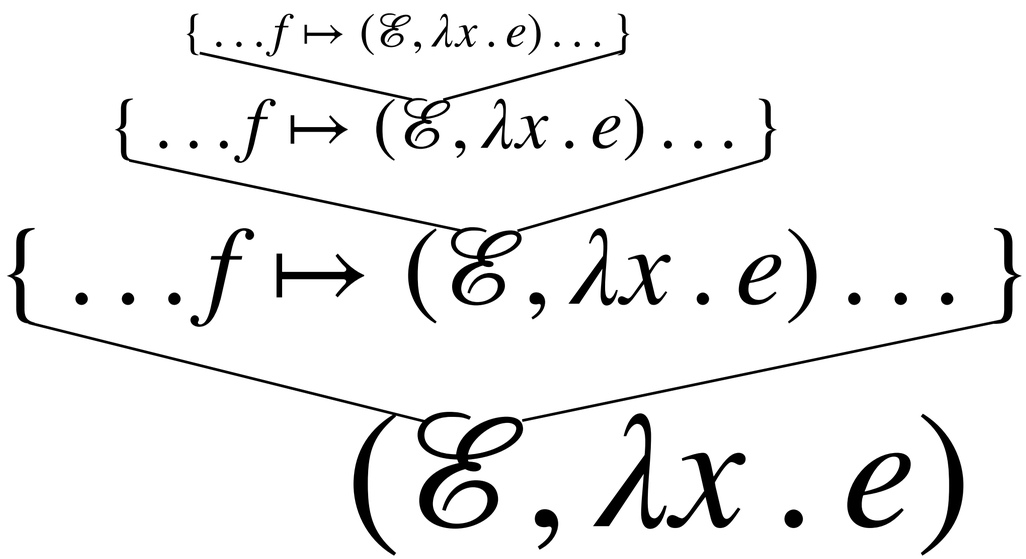
High-Level

```
let f x =
  if x = 0
  then 1
  else f (x - 1)
in f 10
```

We've only considered *non-recursive* function so far

And there's no easy way to do recursion with substitution

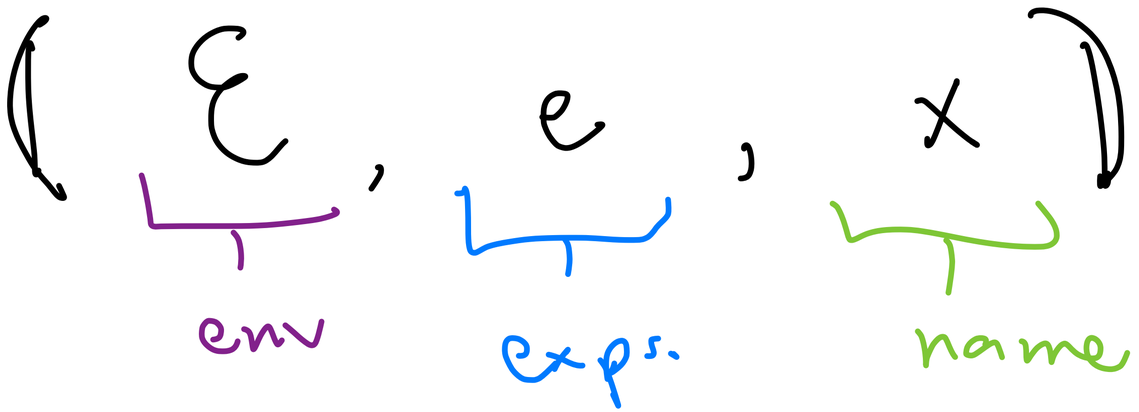
The Problem



In order to implement recursion, a closure has to "*know thyself*"

But we **can't** build infinite structures like this in OCaml

Named Closures



marker for recursive functions

let f = λ x. f x in f true

recursive call

Lambda Calculus^{++} (Syntax, Again)

$$e ::= \lambda x.e \mid ee \mid x \mid n$$
$$\mid \text{if } e \text{ then } e \text{ else } e \mid \text{true} \mid \text{false}$$
$$\mid \text{let } x = e \text{ in } e$$

Semantics Again Again (λ Calculus +)

$e ::= \lambda x.e \mid e\,e \mid x \mid n$
$\mid$ if e then e else $e \mid$ true $\mid$ false
$\mid$ let $x = e$ in e

$v ::= (\!| \mathcal{E}, \lambda x.e |\!) \mid (\!| \mathcal{E}, \lambda x.e, x |\!) \mid n$
$\mid$ true $\mid$ false

$$\frac{\langle \mathcal{E}, e_1 \rangle \Downarrow (\!| \mathcal{E}', \lambda x.e, f |\!) \quad \langle \mathcal{E}, e_2 \rangle \Downarrow v_2 \quad \langle \mathcal{E}'[f \mapsto (\!| \mathcal{E}', \lambda x.e, f |\!)][x \mapsto v_2], e \rangle \Downarrow v}{\langle \mathcal{E}, e_1\, e_2 \rangle \Downarrow v} \text{(appRec)}$$

$$\frac{\langle \mathcal{E}[x \mapsto (\!| \mathcal{E}, \lambda x_1.e_1, x |\!)], e_2 \rangle}{\langle \mathcal{E}, \text{let } x = \lambda x_1.e_1 \text{ in } e_2 \rangle \Downarrow v} \text{(letRec)}$$

(call else the same)

demo